

Executive Summary

This roadmap outlines how to evolve the given 3rd-year HP-folding project into a publishable study. We identify a **Minimal Viable Publishable Contribution (MVPC)** – a concise, novel claim (for example, an improved hybrid CP-RL method or key new insight) that would justify a workshop or short-paper acceptance. We then list specific short-term experiments and enhancements to support that MVPC: extra benchmarks, statistical validation, ablations, and a hybrid algorithmic twist (e.g. using the CP solver to warm-start the RL agent or vice versa). We also cover practical steps (code release, documentation, improved figures) and appropriate target venues (e.g. ML/AI workshops, CP tutorials or student tracks). For each proposed step, we estimate required effort and time. Finally, we include a 2–4 week timeline (mermaid Gantt chart) dividing tasks into immediate (weeks 1–2) vs longer-term.

The MVPC will focus on something like: **“A novel CP-RL hybrid strategy yields superior fold quality or efficiency on benchmark HP sequences.”** To support this claim, we must (1) rigorously reproduce and extend the current results (with multiple random seeds and stronger baselines), (2) implement at least one hybrid or transfer approach, (3) add new data or algorithms so that the contribution is non-trivial, and (4) fully document everything. The plan is broken into immediate (2–4 week) tasks that can be done with modest compute (single GPU + normal CPU) and longer-term tasks (requiring more research or compute). By following this roadmap, the project can reach a defensible scientific contribution suitable for a workshop or student conference.

(1) Minimal Viable Publishable Contribution (MVPC)

- **Core Claim:** For example, *“We introduce a CP-RL hybrid method that accelerates finding near-optimal folds in the 3D HP model”*. This is tighter than “CP vs RL comparison”. It highlights new methodology. Alternatively, if the hybrid is too hard, a strong MVPC could be *“We perform the first thorough statistical comparison of CP-SAT and deep RL on HP folding, revealing key strengths and weaknesses”*. Either way, the MVPC must go beyond what the current project does. It could be:
 - A new hybrid (warm-start CP with RL, or RL-guided CP-search) that demonstrably beats both standalone methods.
 - Or a thorough study that includes multiple runs, statistical tests, and new benchmarks, establishing e.g. “RL eventually matches CP quality but with less compute” – phrased as a useful insight.
- **Defense:** The MVPC should address a clear gap. For instance, the dissertation already notes (p. 38) that combining CP-SAT with LSTM might work. We could concretely try *“pre-training the RL agent on short proteins, then transferring to longer ones”* or *“using CP to generate partial folds as demonstrations for RL”*. Whichever we choose, it must be implementable and testable within weeks. The MVPC’s novelty should be defensible by citing related work (e.g. Espitia et al. 2024 ¹ for pure RL, Mann et al. 2008 ² for CP, but nothing on hybrids).
- **Alignment with Venues:** A MVPC like the above is slim but sufficient for workshop publication. For example, a NeurIPS/ICLR workshop or a CP conference demonstration track might accept “this new hybrid method (with proof-of-concept results)”. The claim must be modest (not a full journal), focused on either algorithm or insight.

(2) Additional Experiments (Table)

Experiment	Purpose	Resources Needed	Success Criteria
Replication: Re-run CP-SAT and RL (LSTM-A) on benchmarks with multiple seeds and extended budgets.	Ensure results are robust; estimate variance.	1–2 GPUs (parallel runs), CPU (CP-SAT); 7h×8=56h per seed for CP; RL ~10h/seq.	Small variance; confirm baseline results qualitatively match.
Statistical Test: Perform t-tests or ANOVA on energies from multiple runs.	Validate claims statistically (not single runs).	Same data as above. Use Python (scipy/stats).	p<0.05 significance for any claimed differences.
Extended Benchmarks: Add 2–4 new HP sequences (varying length/composition) not in original 8.	Test generality beyond original 3d1–3d8.	CPU time (CP-SAT ~7h/seq each), GPU (RL ~10h/seq).	Methods behave similarly on new data (no unexpected failures).
Time-Budget Sweep: Run CP-SAT with shorter/longer limits (e.g. 1h, 10h per seq).	Show how solution quality evolves with time.	CP-SAT solver, use same machine.	Energy vs time curves (e.g., plot of gap vs runtime).
Memory/Scale Ablation: Limit RL agent (smaller LSTM or fewer episodes) or CP (prune moves)	Study sensitivity to compute budget and model size.	Same as above but reduce compute.	Observe trends (e.g. smaller model still works for n<36)
Hybrid Prototype: Implement one hybrid idea (e.g. CP provides top-k folds for RL demonstration, or RL policy biases CP search directions).	Directly add novelty: test if hybrid beats both.	Moderate coding (TensorFlow/PyTorch + CP-SAT API). ~1 GPU + CPU.	Hybrid solution quality > max(CP-only, RL-only) on at least 1 benchmark.
Ablation of Hybrid: Remove components (e.g. no CP demo, or no RL reward) to identify contribution of each part.	Understand which pieces matter.	Re-use hybrid code.	Clear performance drop when a component is removed.

Table: Proposed additional experiments. Each should be documented and ideally automated in code.

Notes on experiments:

- **Datasets:** Use the original 3d1–3d8 sequences plus 2–4 sequences of similar/higher difficulty (e.g. random HP strings or known hard cases from literature).
- **Compute Estimates:** For short sequences ($n \leq 36$), RL training took ~10 hours each on one GPU³; assume similar or more for $n=58$. CP-SAT runs as given (47 min per seq in 7h). For reproducibility, multiple seeds (e.g. 5–10) will multiply GPU time proportionally. Plan usage accordingly (e.g. 2 GPUs in parallel).
- **Metrics:** Report best energy, gap to best-known, runtime until best, and success rate of RL (how often it finds optimum). Statistical significance (t-test) should accompany any claim of superiority.

(3) Methodological Extensions (Novelty)

To **add novelty beyond a simple compare**, we must develop at least one **new method** or twist.

Possible directions:

- **RL Warm-Start via CP (or vice versa):** Use CP-SAT to generate a few high-quality folds or partial folds, and use those as demonstrations or to pre-train the RL agent (e.g. behaviour cloning or imitating the search moves). For example, use HPstruct/CP-SAT to solve short prefixes or subchains optimally, then train the LSTM policy to continue from those. Implementation: after CP finds an optimal structure, treat it as a sequence of moves and use supervised pre-training on the RL network. Expected impact: RL can learn the “easy” part quickly and focus on remaining complexity, improving sample efficiency. This idea is hinted at in literature (e.g. learning from search) ¹.
- **CP Search Guided by RL:** Use the trained RL policy to define a branching heuristic for CP-SAT. In OR-Tools, one can define a search strategy or provide hints (e.g. search with fixed moves from the policy initially). Implementation: e.g., force the first k moves of CP to follow the RL-chosen moves (via strong symmetry-breaking or hinting), then let CP optimize the rest. Expected impact: CP might reach good solutions faster. This is more involved but could be sketched by running CP-SAT with an “assumption” on first moves (if the solver API allows it).
- **Transfer Learning:** Train the RL agent on multiple short, easy sequences (maybe sequences of length 20–30) and then test or fine-tune on longer ones (length 40–58). This could show generalisation ability. The dissertation suggests pre-training on short sequences (p.52). Implementation: run RL on all short sequences together (as multi-task learning), then continue training on longer sequences with the resulting model. Impact: possibly reach better energy or faster on long sequences.
- **Reward Shaping / Curriculum:** Enhance the RL reward or training schedule. For instance, start RL on easy sequences and gradually increase length (curriculum learning). Or shape rewards to emphasize diverse folds or completion. This is more an RL trick than new method but might boost results.
- **Ablation of LSTM-A:** Evaluate how much the multi-head attention helps. If resources, try training the LSTM without attention (or smaller hidden size) to show that the chosen architecture matters. Even if short on time, mention this as future work or minor ablation.

Implementation sketches:

- For each hybrid idea, initially code a prototype on one or two sequences to test feasibility. E.g. write a small script: use OR-Tools to produce e.g. k -step prefix from CP, then modify RL environment to fix those first k moves.
- Use existing codebase (GitHub repo) to add new functions (could be small <200 lines each).
- Measure performance on at least 2 sequences (one short, one long) to justify bigger experiments.

(4) Writing and Reproducibility Tasks

- **Figures/Tables:** Prepare clear visual comparisons. E.g.: (a) Plot best energy vs time for CP vs RL vs hybrid (with error bars); (b) show example conformations (like Fig.3.3 of thesis) comparing

methods; (c) ablation bar charts (hybrid vs components). Ensure all figures have descriptive captions.

- **Code release:** Publish all code (CP models, RL training) on GitHub. Checklist: include `requirements.txt` / `environment.yml`, README with instructions, links to data (HP sequences), scripts to reproduce each experiment. Use literate code (commenting, consistent formatting).
- **Experiment logs:** Save training logs (rewards per episode, solver iterations) and seed parameters. Consider using a tool like Weights & Biases or TensorBoard for visualization; if not, at least plot reward curves (like Fig.3.1 in thesis).
- **Parameter tables:** Document all hyperparameters (learning rate, network size, CP-SAT config) in the paper's appendix or supplementary.
- **Ablation analysis:** Include a short section or appendix table listing how performance changes when removing the hybrid or pre-training components. This strengthens claims.
- **Writing polish:** Ensure the introduction motivates the MVPC (e.g. gap in literature). Condense the background; focus the **narrative on the new idea**. Possibly reframe chapters into "Methods: CP Baseline, RL Baseline, Hybrid Method".
- **Reproducibility statement:** Add a brief section stating code is public and describing compute used (e.g. "Experiments run on [GPU type], seeds fixed, etc.").

(5) Target Venues and Timeline

- **Possible Venues:**
 - *ML/AI Workshops:* NeurIPS/ICLR workshops on AI for science or computational biology (e.g. **ML4PS Workshop**), or ICML/NeurIPS workshops on RL or combinatorial optimisation. These welcome preliminary but well-executed results.
 - *Constraint Programming/OR:* CP (the CP conference) has a Applications track or short paper track; also CPAIOR (CP for AI OR) or INFORMS session on AI. A well-engineered study could fit a short track.
 - *Bioinformatics/Computational Biology:* Unlikely for main tracks; perhaps a student symposium (e.g. BCB student paper) or an MDPI conference on computational biology.
 - *Student/Undergrad journals:* QMUL's internal undergraduate research journal, or venues like UMAP (undergrad ML).

Choose a target that matches paper length and novelty: e.g. a **NeurIPS ML4Bio workshop** or **CP 2026 student paper track**. Workshops may have deadlines mid-year (spring/summer). Assuming April 2026 now, aim for summer 2026 deadlines.

- **Timeline:** Outline of short- vs long-term tasks is given below. The 2–4 week plan is illustrated in the Gantt chart below.

```
gantt
    title 2-4 Week Project Plan
    dateFormat YYYY-MM-DD
    section Weeks 1-2
    Reproduce Experiments      :done,    exp_rep, 2026-04-15, 10d
    Hyperparameter Sweep (CP) :done,    cp_tune, after exp_rep, 5d
    RL Multiple Seeds (Short)  :active,  rl_tune, 2026-04-18, 7d
    section Weeks 3-4
    Implement Hybrid Strategy   :crit,    hybrid, after rl_tune, 7d
```

Run Hybrid Experiments	:	hybrid_run, after hybrid, 7d
Write Draft of Paper	:	writing, 2026-04-22, 14d

Figure: Gantt chart of the 2–4 week plan (Note: dates are illustrative).

(6) 2–4 Week vs Longer-Term Breakdown

Task	Timeline	Effort/Person-hours	Compute	Notes
Weeks 1–2 (Immediate)	Weeks 1–2 (14 days)	~40–60 hrs total	1 GPU, CPU	Quick wins, necessary prep
- Reproduce CP-SAT baseline runs	days 1–5	8 hrs (setup+run)	CPU	Vary time limits to log .
- RL multiple seeds (short seq) ⁴	days 1–10	10×3 hrs = 30 hrs	1 GPU (parallel possible)	Save reward curves.
- Statistical analysis (t-tests)	days 5–7	4 hrs	- (CPU)	Jupyter + SciPy.
- Add 1–2 new short benchmarks	days 6–10	6 hrs setup + run	GPU/CPU as above	Verify at least one new case.
- Prepare improved figures: energy-vs-time plots (with error bars)	days 8–12	8 hrs (plotting)	CPU/GPU data	Matplotlib/ seaborn.
- Outline hybrid method design	days 9–14	4 hrs discussion	-	Decide warm-start vs CP-guided.
Task	Timeline	Effort/Months	Compute	Notes
Months 1–3 (Long-Term)	Weeks 5+ (40+ days)	Substantial (weeks)	Multi-GPU	Deeper R&D needed
- Implement and tune hybrid method	Next ~2-4 weeks	2–3 weeks dev	GPU + CPU	Hardest part; allows tuning.
- Run extended experiments (hybrid vs baseline)	Ongoing	O(10) training runs	Multi-GPU	Gather hybrid results.
- RL transfer learning experiments	Overlapping with above	1–2 weeks	GPU	Train on short, fine-tune long.
- More ablations (architecture, reward)	in parallel	2+ weeks	GPU	If time permits.
- Write and submit paper	1–2 weeks near end	2–3 weeks	-	Draft, revise, figure polish.

Task	Timeline	Effort/Months	Compute	Notes
- Peer review rebuttal prep	After submission	2–3 days	-	Pre-anticipate criticisms.

(7) Risk Assessment & Rebuttal Prep

Risks and counterpoints:

- “*Novelty is limited, just an assignment.*” – Emphasize any new technique (e.g. hybrid) or insight. If only an extensive comparison, frame it as the *first comprehensive study of CP vs DRL on HP* with rigorous analysis. Cite related standards (e.g. “*Prior RL work did not compare to exact solvers*”) to justify novelty.
- “*Results may not show improvement.*” – If the hybrid does not clearly win, still report interesting findings (e.g. it converges faster). Have statistical backing so “no significant difference” is also scientifically interesting. Focus on lessons learned.
- “*Compute fairness.*” – Clearly document CPU/GPU resources so reviewers see fairness (e.g. both methods given comparable search time). In rebuttal, show any improvements persist even under stricter fairness.
- “*Reproducibility.*” – Publish all code and data. In rebuttal, highlight the availability of our GitHub, parameter files, and detailed methodology (so reviewers can verify). Use consistent seeds and report randomness.
- “*Baseline tuning.*” – Ensure baselines (CP-SAT, RL) are well-tuned. In rebuttal, note that we tried multiple solver settings (warm vs default) and tuned hyperparameters via preliminary search. If pressed, provide supplemental tuning details.

By following this plan, the project will add enough new material (method and rigor) to move from a coursework report to a workshop-level publication. Careful implementation, thorough testing, and clear presentation will be key to convincing reviewers of our contribution.

1 [2412.20329] Protein Structure Prediction in the 3D HP Model Using Deep Reinforcement Learning
<https://arxiv.org/abs/2412.20329>

2 (PDF) CPSP tools-Exact and Complete Algorithms for High-throughput 3D Lattice Protein Studies
https://www.researchgate.net/publication/5390121_CPSP_tools-Exact_and_Complete_Algorithms_for_High-throughput_3D_Lattice_Protein_Studies

3 Coarse-Grained Protein Models and Their Applications
<https://pubs.acs.org/doi/10.1021/acs.chemrev.6b00163>

4 [2211.14939] Applying Deep Reinforcement Learning to the HP Model for Protein Structure Prediction
<https://arxiv.org/abs/2211.14939>